the way, it assumed that the application will have a controller named `ProductsController`, containing seven actions with given names.

You can take a look at the routes that were generated for us. We do this by making use of the handy `rails routes` command.

```
      Prefix Verb   URI Pattern
                    Controller#Action
    products GET    /products(.:format)
                    {:action=>"index", :controller=>"products"}
             POST   /products(.:format)
                    {:action=>"create", :controller=>"products"}
 new_product GET    /products/new(.:format)
                    {:action=>"new", :controller=>"products"}
edit_product GET    /products/:id/edit(.:format)
                    {:action=>"edit", :controller=>"products"}
     product GET    /products/:id(.:format)
                    {:action=>"show", :controller=>"products"}
             PATCH  /products/:id(.:format)
                    {:action=>"update", :controller=>"products"}
             DELETE /products/:id(.:format)
                    {:action=>"destroy", :controller=>"products"}
```

All the routes defined are spelled out in a columnar format. The lines will generally wrap on your screen; in fact, they had to be broken into two lines per route to fit on this page. The columns are (optional) route name, HTTP method, route path, and (on a separate line on this page) route requirements.

Fields in parentheses are optional parts of the path. Field names preceded by a colon are for variables into which the matching part of the path is placed for later processing by the controller.

Now let's look at the seven controller actions that these routes reference. Although we created our routes to manage the products in our application, let's broaden this to talk about resources—after all, the same seven methods will be required for all resource-based routes:

index
: Returns a list of the resources.